

ChampionCHIP eXperience — Stage 2 | Design Submission Report



Multicycle RV32I_Zmmul_Xicrc Processor

Team: Equipe 14

Name	University
Hwong Jin Kang (Leader)	UM
Tan Ying Ru	UTHM
Tng Kah Hong	APU
Tan Kai Jian	UPM
Amirul Hakim Bin Abdunnajib	University of Manchester

Table of Contents

1. Project Overview	2
2. Supported Instructions	2
3. Datapath	4
4. Modules and Testbenches	7
5. OpenLane Results	19
6. Firmware Demo	20

Video Presentation URL : https://youtu.be/ja_pg3AdSc0?si=6lZYrneSH67sOOdd

1. Project Overview

The G14 project implements a 32-bit multicycle RISC-V processor supporting the RV32I base instruction set, the Zmmul multiplication extension, and a custom Xicrc CRC extension, targeting the ChampionCHIP Phase 2 design challenge. The processor extends the single-cycle reference core provided in ChampionCHIP training into a 4-state finite-state-machine architecture — FETCH, DECODE, EXECUTE, and WRITEBACK — allowing hardware blocks such as the ALU and the shared instruction/data memory bus to be time-shared across the instruction cycle rather than duplicated, consistent with the resource-sharing philosophy of the reference ChipInventor RVBL-2 core.

The design comprises eleven primary modules, detailed in Section 4: a 3-level hierarchical dual-read-port Register File, an Immediate Generator supporting all five RISC-V immediate formats, a Branch Comparator, a combinational ALU performing eleven operations, a Multiplier implementing all four Zmmul variants (MUL/MULH/MULHSU/MULHU), a CRC unit implementing the Xicrc extension (CRCB/CRCH/CRCW) against the organizer-confirmed CRC-16-CCITT-FALSE algorithm, an Address Decoder routing shared-bus traffic between instruction and data memory, a Load/Store Unit handling all five load and three store instruction variants, and the Control Unit/FSM pair driving the datapath. Every module is individually verified by a self-checking testbench, summarized alongside its functional description in Section 4.

The processor implements 44 of the 44 instructions required for this stage (100% coverage; System/Synchronization instructions were confirmed out of scope by the organizers), verified end-to-end by successfully executing the official Stage 2 validation firmware to completion — all 9 validation stages passed, halting at the correct address with the expected result register value (Section 6). The design was additionally carried through a complete OpenLane physical implementation flow on the sky130A PDK, achieving a clean signoff with zero DRC violations, zero LVS errors, and full setup/hold timing closure at a 625×625 μm die (Section 5).

2. Supported Instructions

2.1 ISA Coverage Table

The implemented processor supports 44 instructions across the RV32I base instruction set, Zmmul multiplication extension, and custom CRC extension. Table 2.1 summarizes the supported instruction categories and implementation coverage.

Category	Expected	Implemented	Coverage
Arithmetic and Logic (Register)	10	10	100%
Arithmetic and Logic (Immediate)	9	9	100%
Load	5	5	100%
Store	3	3	100%
Branch	6	6	100%
Jump	2	2	100%
Upper Immediate	2	2	100%
Multiplication	4	4	100%
CRC	3	3	100%
Total	44	44	100%

Table 2.1

Note: System/Synchronization instructions were excluded from the final supported-instruction scope following the organizer's latest clarification.

Arithmetic and Logic (Register) (10/10): ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND

Arithmetic and Logic (Immediate) (9/9): ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI

Load (5/5): LB, LH, LW, LBU, LHU

Store (3/3): SB, SH, SW

Branch (6/6): BEQ, BNE, BLT, BGE, BLTU, BGEU

Jump (2/2): JAL, JALR

Upper Immediate (2/2): LUI, AUIPC

Multiplication (4/4): MUL, MULH, MULHSU, MULHU

CRC (3/3): CRCB, CRCH, CRCW

2.2 Encoding Table

Opcode / funct3 / funct7 for each implemented instruction, as encoded in the current RTL (Control Unit, ALU, Branch Comparator, LSU). '—' indicates the field doesn't apply to that instruction's format:

Instruction	Opcode	Funct3	Funct7
ADD	0110011	000	0000000
SUB	0110011	000	0100000
SLL	0110011	001	0000000
SLT	0110011	010	0000000
SLTU	0110011	011	0000000
XOR	0110011	100	0000000
SRL	0110011	101	0000000
SRA	0110011	101	0100000
OR	0110011	110	0000000
AND	0110011	111	0000000
ADDI	0010011	000	—
SLTI	0010011	010	—
SLTIU	0010011	011	—
XORI	0010011	100	—
ORI	0010011	110	—
ANDI	0010011	111	—
SLLI	0010011	001	0000000
SRLI	0010011	101	0000000
SRAI	0010011	101	0100000
LB	0000011	000	—
LH	0000011	001	—
LW	0000011	010	—
LBU	0000011	100	—
LHU	0000011	101	—
SB	0100011	000	—
SH	0100011	001	—
SW	0100011	010	—
BEQ	1100011	000	—
BNE	1100011	001	—
BLT	1100011	100	—
BGE	1100011	101	—
BLTU	1100011	110	—
BGEU	1100011	111	—
JAL	1101111	—	—
JALR	1100111	000	—
LUI	0110111	—	—
AUIPC	0010111	—	—

MUL	0110011	000	0000001
MULH	0110011	001	0000001
MULHSU	0110011	010	0000001
MULHU	0110011	011	0000001
CRCB	0110011	000	1000000
CRCH	0110011	001	1000000
CRCW	0110011	010	1000000

3. Datapath

The G14 core implements the multicycle architecture permitted by the challenge rules (Submission Guide, Note 1): the single-cycle RISC-V processor provided in ChampionCHIP training was extended into a 4-state finite-state-machine (FSM) design, so that hardware blocks — most notably the ALU and the instruction/data memory bus — are time-shared across the FETCH, DECODE, EXECUTE and WRITEBACK states rather than duplicated. This is the same design philosophy the Block Guide describes for the reference ChipInventor RVBL-2 core (§2.1, §3.1.1), and it is the reason a single ALU computes arithmetic/logic results, load/store addresses, *and* branch/jump targets, and why one shared address bus serves both IMEM and DMEM.

Figure 3.1 below shows the complete datapath: every functional block described in Section 4, how they are wired together, and — as dashed lines — the control signals the Control Unit drives into each one. Solid arrows are data; dashed arrows are control.

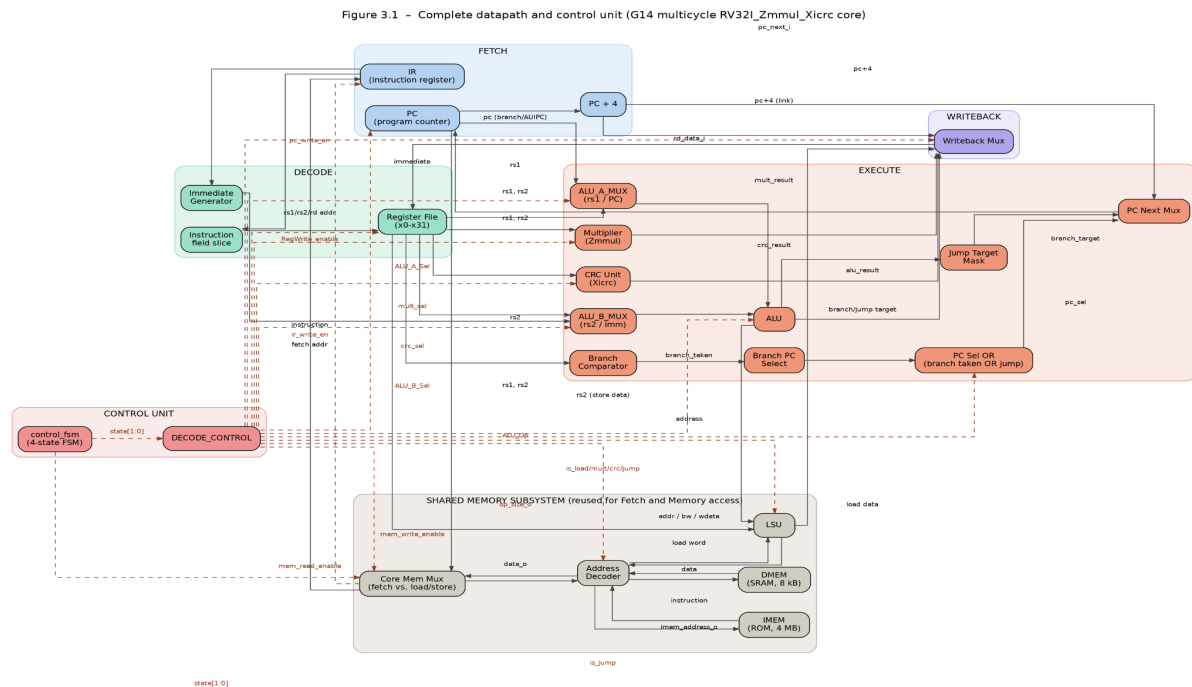


Figure 3.1. Complete datapath and control unit of the G14 core, showing data flow (solid) and control signals (dashed) between all main modules.

3.1 Datapath walkthrough by stage

The FSM (control_fsm) cycles unconditionally through four states, and DECODE_CONTROL re-decodes the same instruction word in every one of them, gating its outputs by state_i so that each block only acts in the state where it is needed.

FETCH (state 00). PC drives its address through CORE_MEM_MUX — the arbiter that decides whether the shared bus is currently carrying the fetch address or an LSU load/store address — and into address_decoder_G14, which routes it to IMEM. The returned instruction word is latched into IR only when CORE_MEM_MUX asserts ir_write_en (true only while the FSM is in FETCH), so IR holds its value through the remaining three states. In parallel, an adder produces PC + 4, which is staged for use both as the default next-PC value and, for JAL/JALR, as the link value written back to rd.

DECODE (state 01). The instruction slice block extracts rs1, rs2, rd, and funct3 directly from IR. rs1/rs2 addresses go to the **Register File**, which returns rs1_data_o/rs2_data_o combinationally (asynchronous read, per Block Guide §3.1.4). The full instruction also feeds the **Immediate Generator**, which combinationally produces the correctly sign-/zero-extended immediate for whichever of the I/S/B/U/J formats applies, and feeds DECODE_CONTROL, which computes every control signal used later in the cycle (ALU_Op, ALU_A_Sel, ALU_B_Sel, is_branch, is_jump, is_load, op_size_o, is_mult, is_crc, mult_sel, crc_sel) up front, even though most of them are only *acted on* in later states.

EXECUTE (state 10). ALU_A_MUX and ALU_B_MUX — external to the ALU itself, per the Block Guide's "external mux" ALU convention — select the ALU's two operands: operand A is rs1 for most instructions or PC for branches, JAL, and AUIPC (ALU_A_Sel); operand B is rs2 for register-register ops or the immediate for everything else (ALU_B_Sel). The **ALU** then performs one of eleven operations selected by ALU_Op (Block Guide Table 9) — this single result is reused three ways depending on instruction type:

- as the arithmetic/logic result (R-type/I-type ALU ops),
- as the effective memory address, forwarded to the **LSU** (loads/stores),
- as the raw jump/branch target, forwarded through **Jump Target Mask** (which clears bit 0, since RISC-V branch/jump targets are always halfword-aligned) into the **PC Next Mux**.

Alongside the ALU, the **Branch Comparator** independently compares rs1/rs2 per funct3 and raises branch_taken; **Branch PC Select** ANDs that with is_branch, and a final OR gate combines this with is_jump (so JAL/JALR also select the computed target, not just taken branches) to decide whether the branch/jump target — rather than PC+4 — should be latched next. The **Multiplier** (Zmmul) and **CRC unit** (Xicrc) run in parallel off the same rs1/rs2 operands, each producing its own result gated by is_mult / is_crc. The **LSU** converts the ALU-computed address plus op_size_o into the byte-write mask and positioned store data (stores) or extracts and sign-/zero-extends the correct byte(s) from the memory word (loads), per Block Guide §3.3 — the same shared bus and address_decoder_G14 used for FETCH now carries this LSU traffic, arbitrated the other way by CORE_MEM_MUX.

WRITEBACK (state 11). The **Writeback Mux** selects among the ALU result, the loaded data from the LSU, the multiplier result, the CRC result, or PC+4 (for JAL/JALR link), based on is_load / is_mult / is_crc / is_jump, and this value is written into the **Register File** at rd when RegWrite_enable is high (asserted only in this state, per §4.11). Simultaneously, **PC Next Mux**'s already-resolved value (branch target, jump target, or PC+4) is latched into PC when pc_write_en is high (also asserted only in WRITEBACK), advancing execution to the next instruction and looping the FSM back to FETCH.

3.2 Control-signal summary

Signal	Driven by	Active when	Effect
ir_write_en	CORE_MEM_MUX	state = FETCH	Latches fetched word into IR
ALU_A_Sel / ALU_B_Sel	DECODE_CONTROL	every state (combinational)	Selects ALU operand sources
ALU_Op	DECODE_CONTROL	every state	Selects ALU function (Table 9)
mem_write_enable	DECODE_CONTROL	state = EXECUTE, store instr.	Enables DMEM write via address decoder
mem_read_enable	DECODE_CONTROL	state = EXECUTE or WRITEBACK, load instr.	Keeps DMEM read data available through commit
RegWrite_enable	DECODE_CONTROL	state = WRITEBACK, result-producing instr.	Commits rd in Register File
pc_write_en	DECODE_CONTROL	state = WRITEBACK	Advances PC
mult_sel / crc_sel	DECODE_CONTROL (= funct3[1:0])	R-type Zmmul/Xicrc	Selects MUL/MULH/... or CRCB/CRCH/CRCW variant

This table, together with Figure 3.1, is what shows the FSM's side effects (memory writes, register commit, PC update) are confined to exactly one state each — the mechanism that lets fully combinational blocks (ALU, LSU, Immediate Generator, Branch Comparator, Multiplier, CRC) be safely re-evaluated every cycle without corrupting architectural state.

3.3 Block Diagrams

Figure 3.1 above is the complete wiring view required for this section. **Figure 3.2** gives the same architecture regrouped by FSM state, useful as a presentation-friendly companion when walking through one instruction's lifecycle for the video demo (Submission Guide §7):

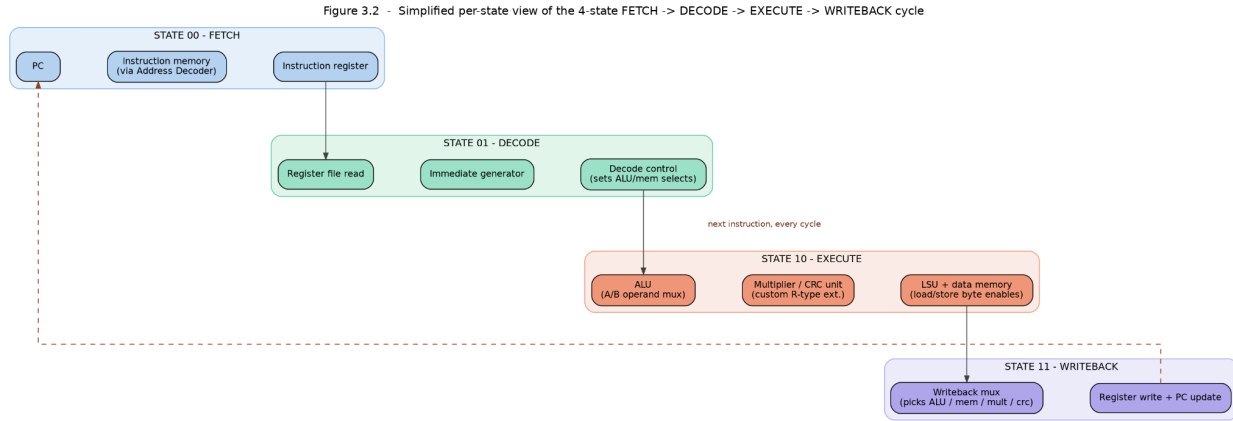


Figure 3.2. Simplified per-state grouping of the same datapath: which blocks are active in FETCH, DECODE, EXECUTE, and WRITEBACK, and the FSM's return to FETCH each cycle

Per-module port tables, signal-level behavior, and testbench evidence for every block shown above (Register File, Immediate Generator, Branch Comparator, CRC Unit, Address Decoder, Multiplier, IMEM, DMEM, LSU, ALU, Control Unit/FSM) are already documented individually in **Section 4**, which this datapath view ties together.

4. Modules and Testbenches

Figure 4.1 presents the complete processor implementation in ChipInventor, showing the interconnections among the developed RTL modules. For clarity, the modules are grouped according to their primary functional roles: fetch, decode, execute, memory access, write-back, control, and PC-update/feedback logic. These regions represent functional groupings within the multi-cycle processor rather than independent pipeline stages.

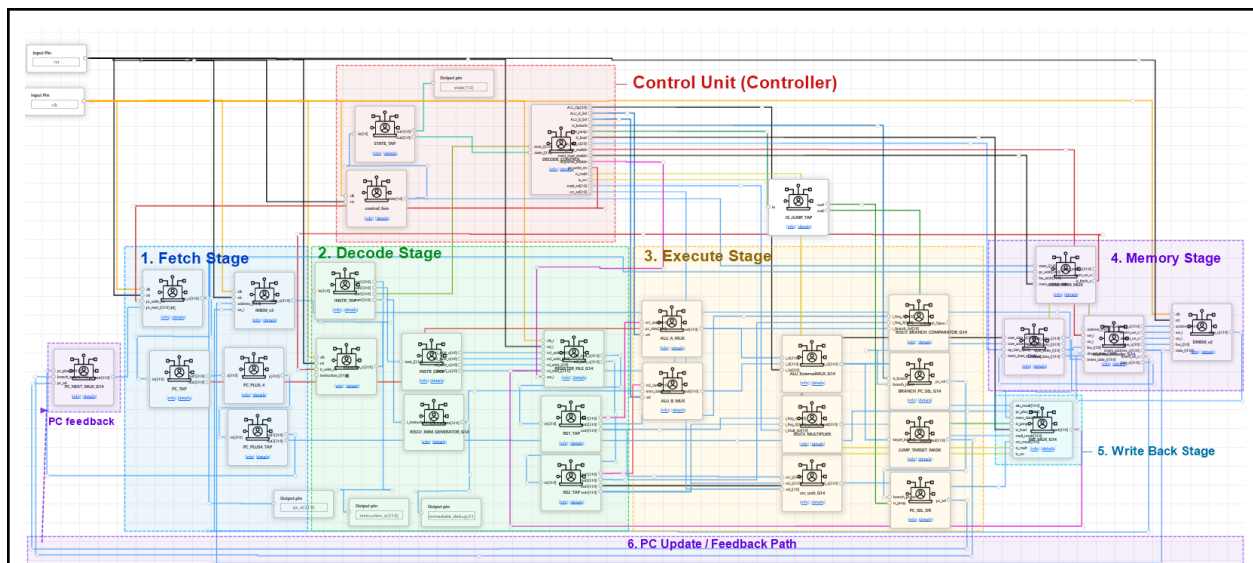


Figure 4.1: Complete architecture with annotated functional regions and module interconnections.

The following subsections describe the functionality, interfaces, implementation, and verification of the major developed modules.

4.1 Register File

Status: Complete — 11/11 testbench assertions passed.

The Register File provides 32 general-purpose 32-bit registers (x0–x31) for the RISC-V processor. It supports two asynchronous read ports and one synchronous write port. Register x0 is protected from writes and therefore remains hardwired to zero.

Ports

Signal	Direction	Width	Description
clk i	in	1	clock
rst i	in	1	reset (synchronous)
rs1_addr i	in	5	selects register for read port 1
rs2_addr i	in	5	selects register for read port 2
rd_addr i	in	5	selects register to write
rd_data i	in	32	value to write into rd
we i	in	1	write enable, from Control Unit
rs1_data o	out	32	value read from rs1
rs2_data o	out	32	value read from rs2

Behavior

The write port and the two read ports use two different internal groupings, rather than indexing `r_Registers` directly by the full 5-bit address in both cases. The write path splits into four 8-register groups (selected by `rd_addr_i[4:3]`), unchanged from an earlier, already-verified design. The two read paths (`rs1_addr_i`, `rs2_addr_i`) each split into eight 4-register groups (selected by `addr[4:2]`, combined through a 2-level mux tree) — restructured this way as part of exploring synthesis-friendly access patterns for physical implementation (Section 5). Both decompositions reconstruct the same 5-bit address, so they always resolve to the same physical register regardless of which one computes a given access.

- Asynchronous read: `rs1_data_o` / `rs2_data_o` update combinatorially as soon as address inputs change (Block Guide §3.1.4: "simultaneous asynchronous reading").
- Synchronous write: `rd_data_i` is written on the rising edge of `clk_i` only when `we_i` is high — matches the Write Back stage (§3.2).
- Write visibility: a value written on a given clock edge is not visible on the read ports until the following cycle.
- x0 protection: only the write path's group 0 (`group_we0`) excludes `rd_addr_i == 0`; the other three write-groups cover addresses 8–31 and never need the exclusion.

Testbench Summary

Self-checking testbench (Icarus Verilog), 11 assertions, all passing — re-run in full after the read path was restructured into the 8-group tree, to confirm the restructuring didn't change behavior:

- Write-then-read correct at x5, x10, x20, and x30 — confirms both the write path's 4-group decoder and the read path's 8-group tree route every tested address correctly.
- x8 — a group boundary under *both* decompositions — read back correctly, confirming neither path mishandles the transition.

- x0 write-block: write attempted with `we_i` high and non-zero data; confirmed x0 still reads 0.
- Dual read: simultaneous read of x5 and x20 returned correct, distinct values — confirms the two read-mux trees don't interfere.
- `we_i` disabled: write attempted with `we_i` low; confirmed target register unchanged.
- Same-cycle / next-cycle timing: old value visible before the clock edge, new value only visible the following cycle.

4.2 Immediate Extension

Status: Complete — all 5 RISC-V immediate formats implemented and tested.

Purpose: Extracts the constant ("immediate") value embedded in a RISC-V instruction and reconstructs it as a sign- or zero-extended 32-bit operand for the ALU. Decodes all five formats the ISA scatters an immediate across: I, S, B, U, and J (Block Guide §3.1.5).

Ports

Signal	Direction	Width	Description
i Instruction	in	32	full instruction word
o Immediate	out	32	extracted, extended immediate value

Behavior — Format Decode

Purely combinational (§3.1.5: "being completely combinational"). Opcode selects which layout applies:

Format	Example instr.	Bit source	Extension
I	ADDI, LW, JALR	instruction[31:20]	sign-extend
S	SW, SB, SH	{[31:25],[11:7]}	sign-extend
B	BEQ, BNE, ...	{[31],[7],[30:25],[11:8],0}	sign-extend
U	LUI, AUIPC	{[31:12], 12'b0}	none — place + zero-fill
J	JAL	{[31],[19:12],[20],[30:21],0}	sign-extend

U-type is not a sign-extension — the raw 20 bits are placed directly into `immediate[31:12]` with the bottom 12 bits zero-filled, since there is no sign bit to extend. B-type and J-type both end in an implicit 0 bit (branch/jump targets are always even), reconstructed from a scrambled bit order rather than a contiguous field.

Testbench Summary

Format	Test values	Result
I-type	-5, +100	0xfffffff, 0x00000064 — both correct
S-type	-20, +40	0xfffffec, 0x00000028 — both correct
B-type	-16, +100	0xfffff0, 0x00000064 — both correct
U-type	0xABCDE, 0x00001	0xabcd000, 0x00001000 — no sign-extension applied
J-type	-2048, +4094	0xffff800, 0x00000ffe — both correct

Test values were built by packing a known signed immediate into the correct scattered bit positions for each format (inverse of the DUT's own decode logic), then comparing against Verilog's own automatic sign-extension of that value — removes manual hex/two's-complement arithmetic as a source of testbench error. B/J-type test values were chosen even (as required) and include boundary values (± 2048 for B-type, $+4094$ for J-type).

4.3 Branch Comparator

Status: Complete — fixed a signed/unsigned comparison bug in ChipInventor's library block, where BLTU and BGEU incorrectly shared BGE's signed-comparison default case.

Purpose: Evaluates the branch condition for conditional jump instructions (BEQ, BNE, BLT, BGE, BLTU, BGEU), comparing rs1 and rs2 and outputting whether the branch should be taken (Block Guide §3.1.6).

Ports

Signal	Direction	Width	Description
i Reg A	in	32	rs1 value (declared signed)
i Reg B	in	32	rs2 value (declared signed)
i Branch Sel	in	3	selects branch type; matches funct3 encoding
o Branch Taken	out	1	1 if the branch condition is true, else 0

Behavior — Branch Type Decode

Sel (binary)	Instruction	Comparison
000	BEQ	$A == B$
001	BNE	$A != B$
010 / 011	—	reserved, unused by any branch instruction
100	BLT	$A < B$, signed
101	BGE	$A >= B$, signed
110	BLTU	$A < B$, unsigned
111	BGEU	$A >= B$, unsigned

Testbench Summary

Branch type	Cases tested	Result
BEQ / BNE	5==5, 5==6, 5!=6	all correct
BLT / BGE	-5 vs 3, 3 vs -5 (signed)	all correct
BLTU	5 vs 10, 10 vs 5 (unsigned)	correct — previously broken (see Issue & Fix)
BGEU	0xFFFFFFFF vs 1, 1 vs 0xFFFFFFFF, 5 vs 10	correct — previously broken; BLTU/BGEU now differentiate correctly

4.4 CRC Unit (Xicrc Extension)

Status: Complete — verified against organizer-provided test vectors.

Purpose: Implements the custom Xicrc extension (crcb, crch, crcw), computing CRC-16 over 1, 2, or 4 bytes of an operand, without needing a dedicated peripheral (Block Guide §3.1.3, Table 8).

Spec Resolution

The Block Guide defines the instruction interface (opcode/funct3/funct7 encoding, R-type format) but does not specify the underlying CRC algorithm. Confirmed directly with organizers:

- Algorithm: CRC-16-CCITT-FALSE
- Polynomial: 0x1021
- Initial value: 0xFFFF (per computation chain, not hardwired into the module)
- No input/output reflection; Final XOR = 0x0000
- rs1 = data, rs2 = seed (supports chained calls — result of one call feeds the next as seed)
- crcb / crch / crcw are not three different algorithms — all three run the same CRC-16-CCITT-FALSE logic, consuming 1 / 2 / 4 bytes of rs1 respectively (MSB-byte-first); the result is always 16 bits, placed in the lower half of rd with the upper 16 bits zero-filled.

Ports

Signal	Direction	Width	Description
rs1_i	in	32	data operand
rs2_i	in	32	seed (current CRC state)
sel_i	in	2	00=crcb, 01=crch, 10=crcw (from funct3[1:0])
rd_o	out	32	CRC result — lower 16 bits, upper 16 bits zero

Behavior

Purely combinational (Block Guide §3.1.3: "without latency"). The current implementation stores all 4 bytes of rs1_i in a shared array (byte_arr[0..3], MSB-first) and runs a single loop over that array, starting at a sel_i-dependent index (start_idx) rather than unrolling three separate call sites: crcb starts at index 3 (only the last byte), crch starts at index 2 (last 2 bytes), crcw starts at index 0 (all 4 bytes). This reframes crcb/crch/crcw as "the same 4-byte chain, entered partway through" rather than three independently-sized chains, which is a more compact way of expressing the same underlying algorithm than the earlier fully-unrolled version.

Testbench Summary

Verified against the organizer's official test program (chained crcb / crch / crcw sequences, expected result 0x1E82 in all three cases) — re-run in full after the implementation was restructured to the shared-array/start-index form, to confirm the restructuring didn't change behavior:

Chain	Inputs (chained, seed=0xFFFF)	Result
crcb x8	0x12,0x34,0x56,0x78,0x90,0xAB,0xCD,0xEF	0x1E82 — PASS
crch x4	0x1234, 0x5678, 0x90AB, 0xCDEF	0x1E82 — PASS
crcw x2	0x12345678, 0x90ABCDEF	0x1E82 — PASS

All three chains converge on the same result, confirming crcb/crch/crcw implement the identical underlying algorithm at different byte granularities, matching the organizer-confirmed spec.

4.5 Address Decoder

Status: Complete — 15/15 self-checking testbench assertions passing (local + ChipInventor).

Purpose: Routes memory accesses between IMEM and DMEM based on address, and multiplexes read data back to the core (Block Guide §4.4, Figure 3). Reused for both instruction fetch and load/store, sharing hardware per the multicycle resource-sharing design.

Ports

Signal	Direction	Width	Description
address_i	in	32	full byte address from core
we_i	in	1	write enable request from core
oe_i	in	1	read (output enable) request from core
bw_i	in	4	byte-write mask from core (for sb/sh)
dmem_data_i	in	32	data read back from DMEM
imem_data_i	in	32	data read back from IMEM
dmem_address_o	out	32	address forwarded to DMEM
dmem_we_o	out	1	write enable to DMEM
dmem_oe_o	out	1	read enable to DMEM
dmem_bw_o	out	4	byte-write mask to DMEM
imem_address_o	out	32	address forwarded to IMEM
imem_oe_o	out	1	read enable to IMEM (no we_o — IMEM is read-only)
data_o	out	32	muxed read data returned to core

Behavior

- Address range detection: `is_imem` / `is_dmem` computed by comparing fixed high-order address bits against the base addresses in Table 13 (`address_i[31:22]==1` for IMEM, `address_i[31:13]` for DMEM) — no full range comparator needed since each region size is a power of two.
- Address forwarding: bottom 2 bits cleared (4-byte alignment) before forwarding; the address value itself is otherwise unchanged (not shifted), consistent with how the D-team's IMEM/DMEM index their internal word arrays.
- `we_i` is only ever forwarded to DMEM, gated by `is_dmem` — IMEM has no `we_o` port at all, so writes to IMEM addresses are silently dropped at both the decoder and the memory.
- `oe_i` is gated independently per device by its own `is_XXX` signal.
- `bw_i` is only meaningful for DMEM; forced to 0 for any non-DMEM access.
- `data_o` is muxed from whichever device `is_XXX` signal is active; defaults to 0 if the address matches neither range.

Testbench Summary

6 scenarios, 15 individual assertions, all passing:

- DMEM read with an unaligned address — confirmed correct snap to word boundary and correct data returned.
- DMEM write with a byte mask — confirmed `we_o` and `bw_o` both reflect the request exactly.
- IMEM read — confirmed correct routing (not DMEM).
- Attempted write to an IMEM address — confirmed `dmem_we_o` stays low (write correctly blocked).
- Address outside both IMEM and DMEM ranges — confirmed neither device enabled, `data_o` defaults to 0.
- DMEM address at the top edge of its valid range — confirmed still correctly recognized.

4.6 Multiplier (Zmmul Extension)

Status: Complete — all four Zmmul operations verified, with 7/7 self-checking testbench cases passing.

Purpose: Implements the Zmmul extension (mul, mulh, mulhsu, mulhu) — the multiplication-only subset of the M extension (Block Guide §2.1, Table 7).

Ports

Signal	Direction	Width	Description
i_Reg_A	in	32	rs1 operand
i_Reg_B	in	32	rs2 operand
i_Mult_Sel	in	2	00=MUL, 01=MULH, 10=MULHSU, 11=MULHU
o_Mult_Result	out	32	result — low word for MUL, high word otherwise

Behavior

Each 32-bit operand is sign- or zero-extended to 64 bits per Table 7's signedness rule for the selected operation, then multiplied as a 64x64 signed product (declared as a 64-bit result). Although a full 64x64 product is mathematically up to 128 bits, the two source operands are each only 32 bits, so the true product magnitude never exceeds 2^{64} — the 64-bit result captures it exactly with no truncation, verified by comparison against a 128-bit reference computation for both the maximum-magnitude signed case and the maximum unsigned case.

MUL selects the low 32 bits of the product; MULH, MULHSU, and MULHU all select the high 32 bits, differing only in how the two operands were sign/zero-extended beforehand.

Testbench Summary

Verified all four Zmmul instructions using extreme boundary values to rigorously test the 64-bit sign/zero extension logic:

- **MUL:** Tested $-1 * 5$. Confirmed correct lower 32-bit truncation (0xFFFFFFFFB).
- **MULH:** Tested $-2147483648 * -2$. Confirmed correct upper 32-bit positive overflow (0x00000001).
- **MULHSU:** Tested -1 (Signed) $* 0xFFFFFFFF$ (Unsigned). Confirmed correct signed-by-unsigned interpretation, generating the correct upper 32-bit negative result (0xFFFFFFFF).
- **MULHU:** Tested $0xFFFFFFFF * 0xFFFFFFFF$ (both Unsigned). Confirmed correct maximum unsigned 64-bit product, yielding the correct upper 32 bits (0xFFFFFFF). All 7/7 assertions passed successfully.

4.7 Instruction Memory (IMEM)

Status: Complete — full validation firmware successfully executed in ChipInventor; a reduced-size IMEM configuration is used for the mock OpenLane synthesis flow.

Purpose: ROM holding the program's instructions and any constant values referenced during execution (Block Guide 4.2). Read-only — nothing in the processor ever writes to it during normal operation.

Ports

Signal	Direction	Width	Description
clk	in	1	clock (present for interface stability, unused internally)
rst	in	1	reset (unused — IMEM is ROM, nothing to reset)

address i	in	32	address to read (4-byte aligned)
oe i	in	1	output enable — gates whether a read occurs
data o	out	32	instruction or constant value at address i

Behavior

- Combinational read — data_o updates in the same simulation instant as address_i/oe_i changes, no clock edge required.
- Read only occurs when oe_i is asserted; data_o outputs 0 when disabled.
- No write capability of any kind, matching the Address Decoder's interface (which never routes a we_o to IMEM).
- The official validation firmware is loaded directly into IMEM per the organizer's Stage 2 firmware repository, by transcribing its instructions into the module. This has been completed and run successfully.
- For synthesis, organizers have confirmed a reduced-size IMEM/DMEM is acceptable, to keep OpenLane runtime manageable — a smaller instruction set is loaded for this "mock synthesis" pass rather than the full validation firmware.

Testbench Summary

- Standalone: confirmed data_o tracks address_i combinatorially (no clock dependency), verified with both increasing and decreasing addresses.
- Standalone: confirmed oe_i=0 forces data_o=0 immediately, and re-enabling oe_i immediately restores the correct value.
- Integration (control_fsm + PC + IMEM + IR): confirmed instruction_o updates correctly at the Fetch-to-Decode edge for consecutive instructions, not one state late.

4.8 Data Memory (DMEM)

Status: Complete — full-size 8 KB DMEM used for functional validation; a reduced-size DMEM configuration is used for the mock OpenLane synthesis flow.

Purpose: SRAM holding program data — variables (.data, .bss, stack, heap) used during execution (Block Guide §4.3). Supports both read and write, at byte granularity.

Ports

Signal	Direction	Width	Description
clk	in	1	clock
rst	in	1	reset (currently unused — see Known Limitations)
address i	in	32	address to access (4-byte aligned)
we i	in	1	write enable
oe i	in	1	output enable — gates whether a read occurs
bw i	in	4	byte-write mask, one bit per byte lane
data i	in	32	data to write
data o	out	32	data read from address i

Behavior

- Synchronous read and write — both complete one clock cycle after being requested, per spec (§4.3: "the operation will only be completed after a clock cycle").
- Write and read are independent — a write does not require oe_i high, and a read does not require we_i high; verified directly by writing with oe_i low and confirming the value on a subsequent read.

- Byte-write mask (bw_i) allows partial-word writes (sb, sh) without disturbing other bytes, implemented as 4 independent conditional writes, one per byte lane.
- For synthesis ("mock OpenLane" pass), a reduced-size variant (16 bytes) is used per organizer guidance, to keep synthesis runtime manageable; full-size (8kB) is used for functional/firmware validation.

Testbench Summary

Verified against Block Guide Table 12's worked example (bytes F1/F2/F3/F4 at DMEM addresses 0x0000-0x0003):

- Full-word write (bw_i=1111) followed by read confirmed correct value (F4F3F2F1).
- Byte-2-only write (bw_i=0100, writing 0x12) confirmed only byte 2 changed, other three bytes untouched (F412F2F1).
- oe_i gating confirmed the same way as IMEM — read blocked while low, correct value appears one cycle after re-enabling.
- Simultaneous write+read-enabled case tested — confirmed no data corruption or race condition when both we_i and oe_i are active in overlapping cycles.

4.9 LSU (Load/Store Unit)

Status: Complete — all 5 load types and 3 store types verified against the spec's worked example.

Purpose: Sits between the core and memory, translating between full-word memory access and the byte/half/word-granular access RISC-V load/store instructions need (Block Guide §3.3): byte selection, sign/zero extension for loads, and byte positioning plus write-mask generation for stores.

Ports

Signal	Direction	Width	Description
core_data_o	in	32	data to store, from core
core_address_o	in	32	target address, from core
op_size_o	in	3	encodes which of 8 load/store instructions (custom encoding)
core_data_i	out	32	loaded data, to core (loads only)
mem_address_i	out	32	address forwarded to memory (passthrough)
byte_write_i	out	4	byte-write mask, to memory (stores only)
mem_data_i	out	32	positioned write data, to memory (stores only)
mem_data_o	in	32	word read back from memory (loads only)

op_size_o Encoding (Design Decision)

The Block Guide never defines this signal's encoding, and it cannot simply pass through RV32I's funct3 directly — LB and SB share the identical funct3 value (000), differing only by opcode, so funct3 alone cannot disambiguate them. The Control Unit derives a custom code from both opcode and funct3 together:

op_size_o	Instruction	op_size_o	Instruction
000	LB	100	LHU
001	LH	101	SB
010	LW	110	SH
011	LBU	111	SW

Behavior

- Purely combinational — no clock, no registers. Required because DMEM already provides the one-cycle synchronous delay the FSM timing model is built around; a registered LSU would stack an extra delay and break the 4-state FSM assumption.

- Store path: generates byte_write_i and shifts core_data_o into the correct byte position based on core_address_o[1:0].
- Load path: extracts the correct byte(s) from mem_data_o based on address, applies sign extension (LB, LH) or zero extension (LBU, LHU), or passes through unmodified (LW).

Testbench Summary

Verified against Block Guide Table 12's worked example (word F4F3F2F1 at DMEM address 0x0000):

Instruction	Test case	Result
LB / LBU / LH / LHU / LW	byte offsets 0 and 2	LB→FFFFFFF3, LBU→000000F3, LH→FFFFFF2F1, LHU→0000F2F1, LW→F4F3F2F1 — all correct
SB	byte 2, writing 0x12	byte_write_i=0100, mem_data_i=00120000 — matches spec's worked example
SH	—	byte write i=0011, mem data i=00001234
SW	—	byte_write_i=1111, mem_data_i=AABBCCDD
SB boundary	byte 3, address=0x10010003	byte_write_i=1000, mem_data_i=99000000 — confirms shift logic across full range

4.10 ALU

Status: Complete — 16/16 testbench assertions passing, all 11 operations verified.

Purpose: Performs all arithmetic/logic operations for R-type and I-type instructions, and is reused to compute memory addresses (loads/stores) and branch/jump targets (Block Guide §3.1.1). A/B operand selection (rs1 vs. PC, rs2 vs. immediate) is done outside this module via separate ALU_A_MUX / ALU_B_MUX blocks — this module only receives the already-selected operands.

Ports

Signal	Direction	Width	Description
i_A	in	32	operand A (declared signed)
i_B	in	32	operand B (declared signed)
i_Sel	in	4	operation selector, matches Table 9 encoding
o_Q	out	32	result (declared signed)

Behavior — Operation Table

i_A, i_B, and o_Q are declared signed by default. \$unsigned(...) is applied only where Table 9 explicitly requires unsigned behavior (SRL, SLTU); SLT and MRS need no cast since signed is already the module's default.

i_Sel	Op	Q =
4'h0	PASS B	B
4'h1	ADD	A + B
4'h2	SUB	A - B
4'h3	AND	A & B
4'h4	OR	A B
4'h5	XOR	A ^ B
4'h6	SLL	A << B[4:0]
4'h7	SRL	unsigned(A) >> B[4:0] (zero-fill)
4'h8	MRS (SRA)	A >>> B[4:0] (sign-extend, since A is already signed)
4'h9	SLT	(A < B) ? 1 : 0 (signed compare)
4'hA	SLTU	(unsigned(A) < unsigned(B)) ? 1 : 0

Testbench Summary

All 11 operations tested with representative and edge-case values, including the signed/unsigned distinctions that are easy to get wrong:

Case	Inputs	Result
ADD (negative operand)	-5 + 3	-2 — correct
SRL vs. MRS on a negative number	-1 >> 4 (logical) / -16 >>> 2 (arithmetic)	SRL zero-fills (0x0FFFFFFF); MRS sign-extends (-4) — correctly differentiated
SLT vs. SLTU on a negative number	-1 < 3	SLT (signed) = 1; SLTU (unsigned, -1 reinterpreted as 0xFFFFFFFF) = 0 — correctly differentiated
default / reserved selector	i_Sel = 4'hF	0 — safe default, no undefined behavior

16 of 16 assertions passed. The SRL/MRS and SLT/SLTU pairs are the highest-risk cases in this module (Verilog silently gives the wrong answer if a signed/unsigned cast is missing or misplaced) and were specifically targeted with negative-operand test cases rather than only positive numbers.

4.11 Control Unit / FSM

Status: Complete — Control Unit and four-state FSM behavior verified with 32/32 self-checking testbench assertions passing.

Purpose: Controls the operation of the RISC-V core via a 4-state Finite State Machine (Block Guide §3.2), decoding each instruction's opcode/funct3/funct7 into the control signals that drive the rest of the datapath. Implemented as two cooperating modules: control_fsm (the state register itself) and DECODE_CONTROL (the combinational decode logic, re-evaluated in every state as the FSM advances).

FSM States (control_fsm)

State	Encoding	Description
FETCH	2'b00	Read the instruction from IMEM at the current PC
DECODE	2'b01	Read rs1/rs2 from the Register File, decode the instruction
EXECUTE	2'b10	ALU (or MULT/CRC) performs the operation; memory address or branch/jump target is computed
WRITEBACK	2'b11	Result is committed to the Register File; PC is updated

Transitions are unconditional and cyclic (FETCH -> DECODE -> EXECUTE -> WRITEBACK -> FETCH), advancing on every clock edge; reset forces the state back to FETCH.

Ports — DECODE_CONTROL

Signal	Direction	Width	Description
instr i	in	32	current instruction word
state i	in	2	current FSM state
ALU_Op	out	4	selects the ALU operation (Table 9 encoding)
ALU_A_Sel	out	1	0=rs1, 1=PC (for branch/JAL/AUIPC target calc)
ALU_B_Sel	out	1	0=rs2, 1=immediate
is_branch / is_jump / is_load	out	1 each	instruction category flags
op_size o	out	3	load/store sub-type, passed to the LSU
mem_write_enable / mem_read_enable	out	1 each	gated by both instruction type and current state
RegWrite_enable	out	1	high only in WRITEBACK, for instructions that produce a result

pc_write_en	out	1	high only in WRITEBACK
is_mult / is_crc	out	1 each	Zmmul / Xicrc instruction flags
mult_sel / crc_sel	out	2 each	sub-operation selector, taken directly from funct3[1:0]

Behavior — Instruction Classification

Purely combinational. Opcode alone distinguishes most instruction categories; R-type is further split into ALU / Zmmul / Xicrc by funct7 (0000000/0100000 = base ALU ops, 0000001 = Zmmul, 1000000 = Xicrc). mult_sel and crc_sel are both taken directly from funct3[1:0], since the corresponding units (Multiplier, CRC) use compatible 2-bit sub-selectors.

- ALU_A_Sel selects PC instead of rs1 for branch targets, JAL, and AUIPC (all of which compute an address relative to PC) — notably excluding JALR, which correctly uses rs1 as its base per the RISC-V spec.
- ALU_B_Sel selects the immediate instead of rs2 for every instruction type except R-type ALU/Zmmul/Xicrc ops, which use two register operands.
- mem_read_enable for load instructions is asserted during both EXECUTE and WRITEBACK. During EXECUTE, the synchronous DMEM performs the requested read at the active clock edge, while keeping the read control asserted through WRITEBACK ensures that the loaded data remains available to the LSU and write-back path when the result is committed to the Register File.
- RegWrite_enable and pc_write_en are both gated to WRITEBACK only, regardless of instruction type — this is what keeps the 4-state cycle's side effects (register commit, PC update) confined to a single, well-defined point per instruction.

Testbench Summary

Verified with real instruction encodings spanning every major category, decoded and cross-checked against RISC-V ISA semantics (not just re-derived from the RTL itself):

Instruction	Encoding tested	Checks performed
ADDI x5,x0,10	0x00A00293	ALU_Op=ADD, ALU_B_Sel=immediate, is_load=0, is_branch=0
ADD x7,x5,x6	0x006283B3	ALU_Op=ADD, ALU_B_Sel=rs2 (register-register), is_mult=0, is_crc=0
LW x1,0(x2)	0x00012083	is_load=1, op_size=LW; mem_read_enable and RegWrite_enable both held correctly through WRITEBACK
SW x1,0(x2)	0x00112023	is_load=0, mem_write_enable=1 only in EXECUTE, op_size=SW, RegWrite_enable never asserted (store produces no register result)
BEQ x1,x2,8	0x00208463	is_branch=1, ALU_A_Sel=PC, ALU_B_Sel=immediate, ALU_Op=ADD (for target address)
JAL x1,4	0x004000EF	is_jump=1, RegWrite_enable=1 in WRITEBACK (link address is written)
MUL x3,x1,x2	0x022081B3	is_mult=1, is_crc=0, mult_sel=00 (matches funct3)
CRCB x3,x1,x2	0x802081B3	is_crc=1, is_mult=0, crc_sel=00 (matches funct3)
State-gating sweep (any instruction)	0x006283B3 across all 4 states	pc_write_en confirmed low in FETCH/DECODE/EXECUTE and high only in WRITEBACK

32 of 32 assertions passed. Test instruction encodings were independently generated from the RISC-V ISA's bit-field layout (not copied from the RTL), so this exercises whether the decode logic matches the actual instruction set semantics rather than only checking internal self-consistency.

5. OpenLane Results

Per organizer guidance, a reduced-size IMEM/DMEM (4 words / 16 bytes) configuration is used for this synthesis pass to keep OpenLane runtime manageable. The full 8 kB DMEM and complete official firmware are used for functional/firmware validation (see §4.7-4.8).

5.1 Area and Density

The physical implementation achieved a total die area of **0.390625 mm²** with a usable core area of 0.369 mm². Following physical placement, clock tree synthesis and routing, the final standard cell core utilization settled at **41.98%** (density of 40110.08 cells/mm²). The flow completed with 0 DRC violations, 0 LVS errors, and full setup/hold timing closure at the 20ns target clock period.

5.2 Final GDSII

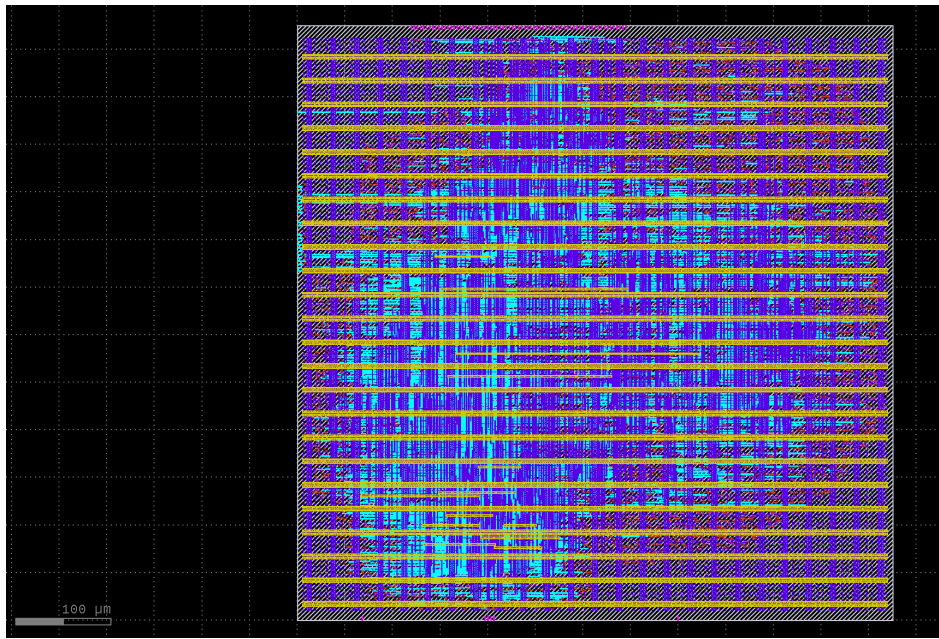


Figure 5.2 : Final GDSII full-die layout of the RISC-V processor (625 x 625 μm), demonstrating successful OpenLane physical synthesis, standard cell placement, and global power/data routing.

6. Firmware Demo

The official validation firmware (per the organizer's Stage 2 firmware repository) has been loaded into IMEM and run successfully in ChipInventor.

6.1 Waveform Evidence

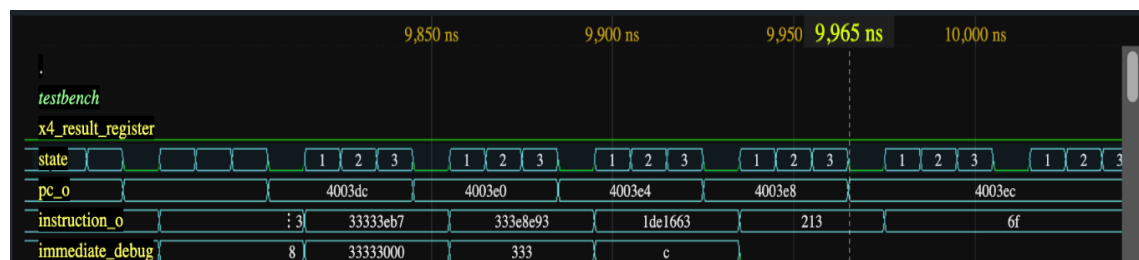


Figure 6.1 : Halt detection at the end of firmware validation.

At `pc_o = 0x004003e8`, the processor executes `li x4, 0` (writing the pass result), then transitions to `pc_o = 0x004003ec` — the `all_good` self-loop (`jal x0, 0`, shown as `instruction_o = 0x6f`). Two complete `FETCH→DECODE→EXECUTE→WRITEBACK` cycles (state values `1→2→3`, repeated) execute at this same address with no further PC change, confirming the processor has reached and is stably held at the intended halt point — not merely paused mid-execution.

6.2 Testbench Log Output

Simulation Log

```
VCD info: dumpfile testbench/testbench.vcd opened for output.
[STAGE 1/9] ALU test entered (pc=0x00400000)
[STAGE 2/9] LSU test entered (pc=0x0040011c)
[STAGE 3/9] Control-flow test entered (pc=0x00400184)
[STAGE 4/9] Register-file test entered (pc=0x00400210)
[STAGE 5/9] Multiplier test entered (pc=0x00400238)
[STAGE 6/9] CRC test entered (pc=0x00400298)
[STAGE 7/9] CRC memory test entered (pc=0x0040034c)
[STAGE 8/9] Arithmetic integration entered (pc=0x00400378)
[STAGE 9/9] Memory transfer test entered (pc=0x00400394)
=====
Validation stages entered: 9 / 9
HALTED after 250 retired instructions at PC=0x004003ec
x4 (result register) = 0x00000000
RESULT: ALL VALIDATION STAGES PASSED
=====
testbench/testbench.v:100: $finish called at 10046000 (1ps)
```

Figure 6.2: Testbench log output, showing stage-by-stage progression and final validation result.

The testbench logs each of the 9 named validation stages the moment execution enters it, the ALU test, LSU test, Control-flow test, Register-file test, Multiplier test, CRC test, CRC memory test, Arithmetic integration, and Memory transfer test. Each stage is tagged with the exact PC address of its entry point, extracted directly from the firmware's own symbol table. All 9 stages report entered (Validation stages entered: 9 / 9). Because the firmware's structure routes any failed check to a shared error handler rather than allowing execution to continue, reaching Stage 9 and then halting at `0x004003ec` (rather than the failure address) is direct evidence that every check in every prior stage passed, not merely that 9 addresses were visited. The final three lines confirm the organizer-specified pass criterion directly: `HALTED after 250 retired instructions at PC=0x004003ec`, `x4 (result register) = 0x00000000`, and `RESULT: ALL VALIDATION STAGES PASSED`.